

4. Driver protocol and instructions

The motor communication is the CAN 2.0 communication interface, the baud rate is 1Mbps, and the extended frame format is adopted as follows:

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	Communication type	data area 2	Destination address	data area 1

The control modes supported by the motor include:

- Operation control mode: set 5 parameters of motor operation control;
- Current mode: the specified Iq current of the given motor;
- Velocity mode: the specified running speed of the given motor;
- Position mode: Given the specified position of the motor, the motor will run to the specified position;

4.1. Description of the communication protocol type

4.1.1. Communication type 0: Get device ID

Gets the device's ID and 64-bit MCU unique identifier

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	0x0	bit15~8: identifies host CAN_ID	target motor CAN_ID	0

Reply frame:

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	0x0	target motor CAN_ID	0xFE	64-bit MCU unique identifier

4.1.2. Communication Type 1: operation control mode motor control instruction

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x1	Byte2: Torque (0~65535) corresponds to (-60Nm~60Nm)	target motor CAN_ID	Byte0~1: target Angle [0~65535] corresponds to $(-4\pi \sim 4\pi)$ Byte2~3: Target angular velocity [0~65535] corresponds to $(-20\text{rad/s} \sim 20\text{rad/s})$ Byte4~5: Kp [0~65535] corresponds to (0.0~5000.0) Byte6~7: Kd [0 to 65535] corresponds to the above data (0.0 to 100.0). After the conversion, the high byte is in front and the low byte is in

Response frame: Response motor feedback frame (see communication type 2)

Communication Type 2: motor feedback data

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	0x2	Bit8~Bit15: CAN ID of the current motor bit21~16: fault information (0 none 1 has) bit21: uncalibrated bit20: Uncalibrated bit20: Gridlock overload fault bit19: magnetic coding fault bit18: overtemperature bit17: Three-phase overcurrent fault bit16: undervoltage fault bit22~23: Mode status 0: Reset mode [reset] 1: Cali mode [calibration] 2: Motor mode [Run]	host CAN_ID	Byte0~1: The current Angle [0~65535] Corresponding to $(-4\pi \sim 4\pi)$ Byte2~3: Current angular velocity [0~65535] corresponds to $(-20\text{rad/s} \sim 20\text{rad/s})$ Byte4~5: Current torque [0~65535] corresponds to $(-60\text{Nm} \sim 60\text{Nm})$ Byte6~7: Current temperature: Temp(Celsius) *10 If the value is higher than 10, the high byte is first and the low byte is last

4.1.3. Communication Type 3: Motor enabled to run

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	0x3	bit15~8: identifies the main CAN_ID	and target motor CAN_ID	

Response frame: Response motor feedback frame (see communication type 2)

4.1.4. Communication Type 4: Motor stops running

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x4	bit15~8: used to identify the main CAN_ID	target motor CAN_ID	When the motor is running normally, 0 must be cleared in the data field. Byte[0]=1: The fault is cleared.

Response frame: Response motor feedback frame (see communication type 2)

Version number read frame

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x4	bit15~8: used to identify the main CAN_ID	target motor CAN_ID	Byte[0]=0x00 Byte[1]=0xC4

Response frame:

data field	29 位 ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x2	Bit8~Bit15: CAN ID of the current motor bit21~16: fault information (0 none 1 has) bit21: uncalibrated bit20: Uncalibrated bit20: Gridlock overload fault bit19: magnetic coding fault bit18: overtemperature	target motor CAN_ID	Byte0=0x00 Byte1=0xC4 Byte2=0x56 Byte3~6: Motor version number Order from high to low

		bit17: Three-phase overcurrent fault bit16: undervoltage fault bit22~23: Mode status 0: Reset mode [reset] 1: Cali mode [calibration] 2: Motor mode [Run]		
--	--	---	--	--

4.1.5. Communication type 6: Set motor mechanical zero

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x6	bit15~8: Identifies the main CAN_ID	and target motor CAN_ID	Byte[0]=1

Response frame: Response motor feedback frame (see communication type 2)

Communication type 7: Set motor CAN ID

Change the current motor CAN ID, effective immediately.

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x7	bit15~8: used to identify main CAN_ID Bit16~23: preset CAN_ID	Target motor CAN ID	

Answer frame: Answer motor broadcast frame (see communication type 0)

4.1.6. Communication type 17: Single parameter read

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x11	bit15~8: Used to identify the main CAN ID	target motor CAN_ID	Byte0~1: index. For details, see the readability parameter table below Byte2~3:00 Byte4~7: In data above 00, the low byte is first and the high byte is second (

Reply frame:

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x11	bit15~8: indicates that the master CAN_ID Bit23~16:00 indicates that the master CAN_ID is successfully read. 01 indicates that the master can_ID	Byte0~1: Byte2~3:00 Byte4~7:Parameter data. 1 byte of data above Byte4 is preceded by low bytes and followed by high bytes at	

4.1.7. Communication type 18: Single parameter write (lost in power failure)

With type 22, the parameter starting with function code 0x20 of the parameter table in the upper computer module can be saved

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x12	bit15~8: Used to identify the main CAN_ID	target motor CAN_ID	Byte0~1: index. For details, see the readability parameter table below Byte2~3: 00 Byte4~7: Parameter data In the preceding data, the low byte is in the front and the high byte is in the rear

Response frame: Response motor feedback frame (see communication type 2)

4.1.8. Communication type 21: Fault feedback frame

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x15	bit15~8: motor	identifies the main	Byte0~3: fault value (non-0: faulty; 0: faulty). Normal)

		CAN_ID	CAN_ID	Bit 16: A-phase current sampling overcurrent Bit 14: Motor stall overload algorithm protection Bit 9: Position initialization fault Bit 8: Hardware identification fault Bit 7: Encoder uncalibrated: Motor encoder not calibrated Bit 5: C-phase current sampling overcurrent Bit 4: B-phase current sampling overcurrent bit3: overvoltage fault bit2: undervoltage fault bit1: driver chip fault bit0: motor overtemperature fault, Default 145 ° C Byte4~7: warning Value bit0: motor overtemperature warning, the default is 135 ° c
--	--	--------	--------	--

4.1.9. Communication type 22: Motor data save frame

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
Description	0x16	bit15~8: identifies the main CAN_ID	and target motor CAN_ID	01 02 03 04 05 06 07 08

Response frame: Response motor feedback frame (see communication type 2)

4.1.10. Communication type 23: Motor baud rate modification frame (re-power-on effect)

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x17	bit15~8: used to identify the main CAN_ID	target motor CAN_ID	01 02 03 04 05 06 F_CMD Among them, the F_CMD byte is the motor baud rate Among them, 01 is 1M 02 is 500K 03 is 250K 04 is 125K

Response frame: Response motor feedback frame (see communication type 0)

4.1.11. Communication type 24: The motor actively reports frames

data field	29-bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x18	bit15~8: identifies the main CAN_ID	target motor CAN_ID	01 02 03 04 05 06 F_CMD Among them, the F_CMD byte is the motor reporting switch 00 is to disable active reporting (default) 01 To enable active reporting, the default reporting interval is 10ms

Response frame:

data field	29 位 ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x18	Bit8~Bit15: CAN ID of the current motor bit21~16: fault information (0 none 1 has) bit21: uncalibrated bit20: Uncalibrated bit20: Gridlock overload fault bit19: magnetic coding fault bit18: overtemperature bit17: Three-phase overcurrent fault bit16: undervoltage fault bit22~23: Mode status 0: Reset mode [reset] 1: Cali mode [calibration] 2: Motor mode [Run]	target motor CAN_ID	Byte0~1: The current Angle [0~65535] Corresponding to (-4π ~4π) Byte2~3: Current angular velocity [0~65535] corresponds to (- 20rad/s~20rad/s) Byte4~5: Current torque [0~65535] corresponds to (- 60Nm~60Nm) Byte6~7: Current temperature: Temp(Celsius) *10 If the value is higher than 10, the high byte is first and the low byte is last

4.1.12. Communication type 25: Motor protocol modification frame (re-power-on effect)

Data field	29 bit ID			8Byte data field
Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
description	0x19	bit15~8: used to identify the main CAN_ID	target motor CAN_ID	01 02 03 04 05 06 F_CMD Among them, the F_CMD byte is the motor protocol type Among them, 0 is a private protocol (default) 1 is the Canopen protocol 2 is the MIT protocol

Response frame: Response motor feedback frame (see communication type 0)

4.1.13. Read and write a single parameter list

index		Description	Type	Number of bytes		R/W Read and write permission
0X7005	run_mode	0: operation mode 1: position mode (PP) 2: Velocity mode 3: Operation mode Current mode 5: Position mode (CSP)	uint8	1		W/R
0X7006	iq_ref	Current mode Iq command	float	4	-43 to 43A	W/R
0X700A	spd_ref	Rotational Velocity mode Rotational speed command	float	4	-20to 20rad/s	W/R
0X700B	limit_torque	torque limit	float	4	0 to 60Nm	W/R
0X7010	cur_kp	Kp	float	4	The default value is 0.17	W/R
0X7011	cur_ki	Ki	float	4	The default value is 0.012	W/R
0X7014	cur_filt_gain	filt_gain	float	4	0 to 1.0, The default value is 0.1	W/R
0X7016	loc_ref	Position Mode Angle instruction	float	4	rad	W/R
0X7017	limit_spd	Location mode (CSP) speed limit	float	4	0 to 20rad/s	W/R
0X7018	limit_cur	Velocity position mode Current limitation	float	4	0 to 43A	W/R
0x7019	mechPos	Mechanical Angle of the loading coil	float	4	rad	R
0x701A	iqf	iq Filter	float	4	-43 to 43A	R
0x701B	mechVel	Speed of the load	float	4	-20 to	R

index		Description	Type	Number of bytes		R/W Read and write permission
					20rad/s	
0x701C	VBUS	Bus voltage	float	4	V	R
0x701E	loc_kp kp	at	float	4	The default value is 60	W/R
0x701F	spd_kp	Indicates the speed kp	float	4	The default value is 6	W/R
0x7020	spd_ki	ki	float	4	The default value is 0.02	W/R
0x7021	spd_filt_gain	Speed filter value	float	4	The default value is 0.1	W/R
0x7022	acc_rad	velocity mode acceleration	float	4	The default value is 20rad/s^2	W/R
0x7024	vel_max	Location mode (PP) speed	float	4	The default value is 10rad/s	W/R
0x7025	acc_set	Location mode (PP) acceleration	float	4	The default value is 10rad/s^2	W/R
0x7026	EPScan_time	Indicates the report time. 1 indicates 10ms. Plus 1 increments by 5ms	uint16	2	The default value is 1	W/R
0x7028	canTimeout	can The timeout threshold, 20000 is 1s	uint32	4	The default value is 0	W/R
0x7029	zero_sta	Indicates the zero flag bit, 0 means $0-2\pi$ and 1 means $-\pi-\pi$	uint8	1	The default is 0	W/R
0x702A	damper	Damping switch	uint8	1	The default value is 0	W/R
0x702B	add_offset	Zero offset	float	4	The default is 0	W/R
0x702C	alveolous_open	Cogging Compensation Switch	uint8	1	The default is 0	W/R

index		Description	Type	Number of bytes		R/W Read and write permission
0x702D	iq_test	Motor Initialization Calibration Switch	uint8	1	The default is 0	W/R
0x702E	dcc_set	Deceleration in Motor PP Mode (Normally Disabled)	float	4	The default is 10rad/s ²	W/R

4.1.14. Read example:

Take reading loc_kp as an example:

Read instruction is

Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
	0x11	0x00FD	0x7F	1E 70 00 00 00 00 00 00
Description	Type 17	Host id 0xFD	Target motor CAN_ID 7F	Byte0~1: index, corresponding to loc_kp

The feedback instruction is

Size	Bit28~bit24	bit23~8	bit7~0	Byte0~Byte7
	0x11	0x007F	0xFD	1E 70 00 00 00 00 F0 41
Description	Type 17	bit15~8: Target motor CAN_ID 7F	Host id 0xFD	Byte0~1: index, corresponding to loc_kp Byte4~7: loc_kp value 30, high right byte, (32-bit single precision) hexadecimal IEEE-754 standard floating point number

4.2. Motor Function Description

(If the following features are unavailable, please upgrade to the latest version via the official Git repository.)

4.2.1. Active Reporting

- **Disabled by default.** Enable via **Type 24.**

- Report type: **Type 2** (default interval: **10ms**). Adjust interval by modifying `EPScan_time` via **Type 18**.
-

4.2.2. Zero-Point Flag (`zero_sta`)

- Modify via:
 - **Host computer**
 - **Type 18** (*requires saving via **Type 22** for communication*)
 - **Default flag:** 0 → Power-on position range: **0–2 π** .
 - **If set to 1:** Power-on position range: **π – π** .
-

4.2.3. Type 2 Update

- Updated to **periodic looping within -4 π –4 π** (enables cycle counting).
- **Note:** Position interface parameters must be adjusted:
 - `P_MIN`: **12.57f**
 - `P_MAX`: **12.57f**

4.2.4. Protocol Switching (*Requires CAN adapter*)

- Methods:
 - Modify `protocol_1` via host computer.
 - Send **Type 25** command.
- **Reboot required** after switching.
- **Post-switch CAN commands:**
 - **CANopen:** Send extended frame (protocol switch frame).
 - **MIT Protocol:** Send standard frame (**Command 8**).

4.2.5. Motor Post-Power-Off Anti-Backdrive Protection

- The motor's post-power-off anti-backdrive protection can be enabled or disabled. If the motor is rotated rapidly while unpowered, a certain damping force will be felt. This protection prevents surges caused by excessively high external rotation speeds. The protection can be disabled by setting damper to 1.

4.2.6. Zero Calibration Rules

- Zero calibration is supported in CSP and Motion Control modes, but blocked in PP mode.
- In old firmware versions, zero calibration would cause a large deviation between the motor's target and actual position, triggering an immediate movement of the motor to the target position.
- In new firmware versions, when zero calibration is performed in CSP or Motion Control mode, the motor's target position is instantly updated to 0, so the motor remains stationary. Zero calibration is unavailable in PP mode.

4.2.7. Position Offset

- A position offset can be added. If an offset of 1 is set, the new zero point will be the original zero point plus 1.
- Some customers cannot perform zero calibration at the physical zero position due to mechanical limits. For example, if the zero point is set at a position of 1 radian, and `add_offset` is then set to 1, after a power cycle, the motor's position of 1 radian will correspond to the original 1 radian position, and the motor's zero point will be 1 radian counterclockwise from the original 1 radian position.

4.2.8. The CANopen ID of the motor is consistent with the CAN ID.

- The CANopen ID is kept the same as the CAN ID under the private protocol.

4.2.9. Motor Initialization Calibration

- When the motor sets `iq_test` to 1, the initialization time is extended, resulting in a more accurate motor initial reference.

4.2.10. Cogging Calibration Function

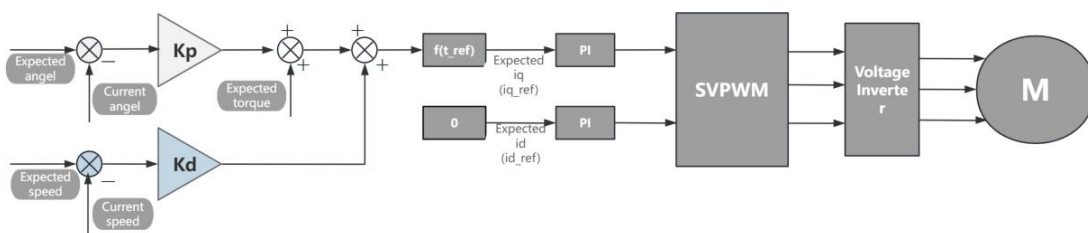
- Before calibration, set `iq_test` to 1 and restart the motor; the motor must be under no-load conditions. Then perform cogging calibration via the host computer. After successful calibration, `max_alve` will be non-zero. Next, set `alveolous_open` to 1 to activate the function in Motion Control mode.

4.2.11. PP Mode Deceleration

By setting `acc_status` to 1, the acceleration and deceleration for PP mode can be configured separately. The deceleration value is stored at 0x702E. When `acc_status` is set to 0, the acceleration parameter controls both acceleration and deceleration.

4.3. Control mode instructions

4.3.1. Operation control mode

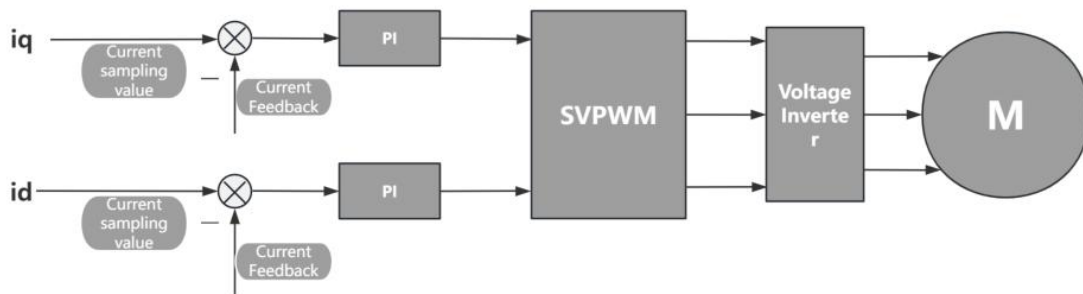


The motor is in operation control mode by default after power-on.

Send motor Enable Run frame (communication type 3) --> Send operation mode motor control command (communication type 1) --> Receive motor feedback frame (communication type 2)

Operation control mode description: The control logic of the operation and control mode is $t_{ref} = K_d * (v_{set} - v_{actual}) + K_p * (p_{set} - p_{actual}) + t_{ff}$. Tref is converted to the expected iq current through an internal formula and output through the current loop Simple control demonstration: Set t_{ff} to 0, v_{set} to 1, K_d to 1, p_{set} to 0, K_p to 0. If there is no external load on the motor, it will run at a speed of 1rad/s. If there is an external load, k_d needs to be increased to resist the external load Set t_{ff} to 0, v_{set} to 0, K_d to 1, p_{set} to 0, K_p to 0, the motor is in damping mode. When the motor is externally rotated, a damping is applied, which increases with the increase of k_d . It should be noted that the motor generates electricity under this condition and requires power supply to prevent overvoltage Set t_{ff} to 0, v_{set} to 0, K_d to 1, p_{set} to 5, K_p to 1. If there is no external load on the motor, it will run to the target position of 5. Increasing k_p will increase the force required to maintain the target position, and k_d is damping. Without k_d , the motor will sway to the target position

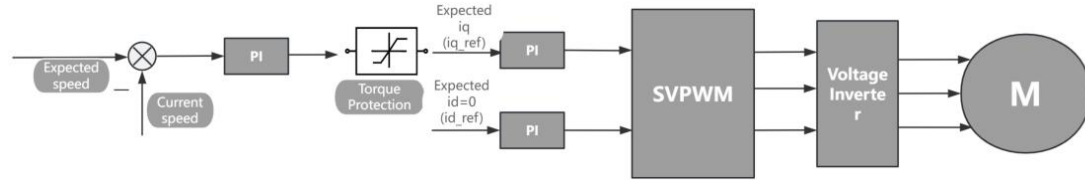
4.3.2. Current mode



Send motor mode parameter write command (communication type 18) Set the runmode parameter to 3 --> Send motor Enable run frame (communication type 3) --> Send motor mode parameter write command (communication

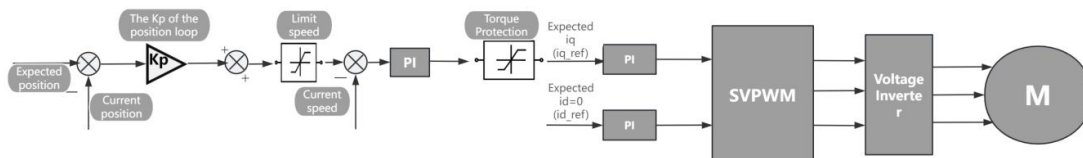
type 18) set the `iq_ref` parameter to the default current instruction

4.3.3. Velocity mode



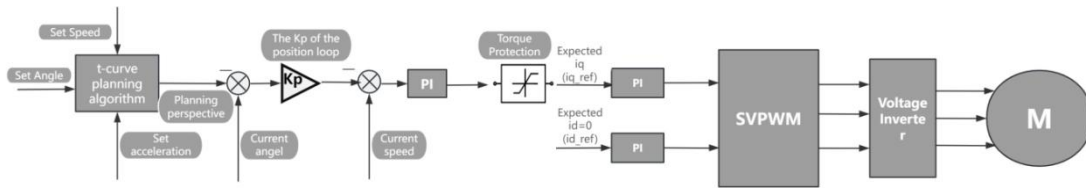
Send motor mode parameter write command (communication type 18) Set the runmode parameter to 2 --> Send motor Enable run frame (communication type 3) --> Send motor mode parameter write command (communication type 18) set `limit_cur` parameter as default maximum current instruction --> Send motor mode parameter write command (communication type 18) Set `acc_rad` parameter as default acceleration instruction --> Send motor mode parameter write command (communication type 18) Set `spd_ref` parameter as default speed instruction

4.3.4. Location Mode (CSP)



Send motor mode parameter write command (communication type 18) Set the runmode parameter to 5 --> Send motor Enable run frame (communication type 3) --> Send motor mode parameter write command (communication type 18) set `limit_spd` parameter as default maximum speed instruction --> Send motor mode parameter write command (communication type 18) Sets `loc_ref` parameter as default position instruction

4.3.5. Location Mode (PP)



Send motor mode parameter write command (communication type 18) Set the runmode parameter to 1 --> Send motor Enable run frame (communication type 3) --> Send motor mode parameter write command (communication type 18) set The vel_max parameter is the default maximum speed instruction --> Send motor mode parameter write command (communication type 18) Set the acc_set parameter to the default acceleration instruction --> Send motor mode parameter write command (communication type 18) Set the loc_ref parameter to the default position instruction

Note: This mode does not support changing the speed and acceleration during operation. If you want to make an emergency stop, you can change vel_max to 0 during the process, and it will stop at the current speed and acceleration plan

4.3.6. Stop running

Sending motor stop frame (communication type 4)

4.4. Program sample

Examples of various mode control motors are provided below (take gd32f303 as an example)

The following are library, function, and macro definitions for the various instances

Plain Text

```
#define P_MIN -12.57f

#define P_MAX 12.57f

#define V_MIN -20.0f

#define V_MAX 20.0f

#define KP_MIN 0.0f

#define KP_MAX 5000.0f

#define KD_MIN 0.0f

#define KD_MAX 100.0f

#define T_MIN -60.0f

#define T_MAX 60.0f

struct exCanIdInfo{

uint32_t id:8;

uint32_t data:16;

uint32_t mode:5;

uint32_t res:3;
```

```
};

can_receive_message_struct rxMsg;

can_trasnmit_message_struct txMsg={

.tx_sfid = 0,

.tx_efid = 0xff,

.tx_ft = CAN_FT_DATA,

.tx_ff = CAN_FF_EXTENDED,

.tx_dlen = 8,

};

#define txCanIdEx (*((struct exCanIdInfo*)&(txMsg.tx_efid)))

#define rxCanIdEx (*((struct exCanIdInfo*)&(rxMsg.rx_efid))) //将扩展帧 id 解析为自定义数据结构

int float_to_uint(float x, float x_min, float x_max, int bits){

float span = x_max - x_min;

float offset = x_min;

if(x > x_max) x=x_max;
```

```
else if(x < x_min) x= x_min;

return (int) ((x-offset)*((float)((1<<bits)-1))/span);

}

#define can_txd() can_message_transmit(CAN0, &txMsg)

#define can_rxd() can_message_receive(CAN0, CAN_FIFO1, &rxMsg)
```

The following lists the common types of communication sent:

4.4.1. Motor Enabled Run frame (communication type 3)

Plain Text

```
void motor_enable(uint8_t id, uint16_t master_id)
{
    txCanIdEx.mode = 3;
    txCanIdEx.id = id;
    txCanIdEx.res = 0;
    txCanIdEx.data = master_id;
    txMsg.tx_dlen = 8;
    txCanIdEx.data = 0;
    can_txd();
}
```

4.4.2. Operation control mode Motor control instruction (communication type 1)

Plain Text

```
void motor_controlmode(uint8_t id, float torque, float MechPosition, float speed, float kp, float kd)
```

```
{
```

```
txCanIdEx.mode = 1;
```

```
txCanIdEx.id = id;
```

```
txCanIdEx.res = 0;
```

```
txCanIdEx.data = float_to_uint(torque,T_MIN,T_MAX,16);
```

```
txMsg.tx_dlen = 8;
```

```
txMsg.tx_data[0]=float_to_uint(MechPosition,P_MIN,P_MAX,16)>>8;
```

```
txMsg.tx_data[1]=float_to_uint(MechPosition,P_MIN,P_MAX,16);
```

```
txMsg.tx_data[2]=float_to_uint(speed,V_MIN,V_MAX,16)>>8;
```

```
txMsg.tx_data[3]=float_to_uint(speed,V_MIN,V_MAX,16);
```

```
txMsg.tx_data[4]=float_to_uint(kp,KP_MIN,KP_MAX,16)>>8;
```

```
txMsg.tx_data[5]=float_to_uint(kp,KP_MIN,KP_MAX,16);
```

```
txMsg.tx_data[6]=float_to_uint(kd,KD_MIN,KD_MAX,16)>>8;

txMsg.tx_data[7]=float_to_uint(kd,KD_MIN,KD_MAX,16);

can_txd();

}
```

4.4.3. Motor mode parameter write command (communication type 18, running mode switch)

Plain Text

```
void motor_reset(uint8_t id, uint16_t master_id)
```

```
{
```

```
txCanIdEx.mode = 4;
```

```
txCanIdEx.id = id;
```

```
txCanIdEx.res = 0;
```

```
txCanIdEx.data = master_id;
```

```
txMsg.tx_dlen = 8;
```

```
for(uint8_t i=0;i<8;i++)
```

```
{
```

```
txMsg.tx_data[i]=0;

}

can_txd();

}
```

4.4.4. Motor mode parameter write command (communication type 18, running mode switch)

Plain Text

```
uint8_t runmode;

uint16_t index;

void motor_modechange(uint8_t id, uint16_t master_id)

{

txCanIdEx.mode = 0x12;

txCanIdEx.id = id;

txCanIdEx.res = 0;

txCanIdEx.data = master_id;

txMsg.tx_dlen = 8;
```

```
for(uint8_t i=0;i<8;i++)

{

txMsg.tx_data[i]=0;

}

memcpy(&txMsg.tx_data[0],&index,2);

memcpy(&txMsg.tx_data[4],&runmode, 1);

can_txd();

}
```

4.4.5. Motor mode parameter write command (communication type 18, control parameter write)

Plain Text

```
uint16_t index;

float ref;

void motor_write(uint8_t id, uint16_t master_id)

{
```



```
txCanIdEx.mode = 0x12;

txCanIdEx.id = id;

txCanIdEx.res = 0;

txCanIdEx.data = master_id;

txMsg.tx_dlen = 8;

for(uint8_t i=0;i<8;i++)

{

txMsg.tx_data[i]=0;

}

memcpy(&txMsg.tx_data[0],&index,2);

memcpy(&txMsg.tx_data[4],&ref,4);

can_txd();

}
```